

ANHANGUERA-LONDRINA/PR

ENGENHARIA DE SOFTWARE

PORTFÓLIO - RELATÓRIO DE AULA PRÁTICA:

NOME DA DISCIPLINA: INFRAESTRUTURA ÁGIL

Londrina/PR
2026

PORTFÓLIO - RELATÓRIO DE AULA PRÁTICA

NOME DA DISCIPLINA: INFRAESTRUTURA ÁGIL

Trabalho de portfólio apresentado como requisito parcial para a obtenção de pontos para a média semestral.

Orientador: Tutor

SUMÁRIO

1. INTRODUÇÃO	**4**
2. MÉTODOS E AMBIENTE	**5**
3. REPOSITÓRIO GIT	**6**
4. DEFINIÇÃO DO PIPELINE	**7**
4.1 JOBS E ESTÁGIOS	**8**
5. EXECUÇÃO COM SUCESSO	**9**
5.1 BUILD E ENTREGA	**10**
6. SIMULAÇÃO DE FALHA	**11**
7. APLICAÇÃO RESULTANTE	**12**
8. ANÁLISE DOS RESULTADOS	**13**
9. CONCLUSÃO	**14**

1. INTRODUÇÃO

Este relatório apresenta a simulação de um pipeline de Integração Contínua para uma aplicação de Loja Virtual. A atividade utilizou Git para versionamento, um arquivo **.gitlab-ci.yml** para descrever os jobs e Docker para construir um artefato reproduzível.

Integração Contínua é a prática de integrar alterações com frequência e submetê-las a verificações automatizadas. Quando um commit é enviado ao repositório, o pipeline executa etapas previamente definidas. Se todas forem aprovadas, a alteração pode avançar para entrega; se uma delas falhar, o fluxo é interrompido e o commit é reprovado.

A prática foi executada localmente no macOS, preservando a lógica do GitLab CI/CD sem exigir a criação de contas externas. Foram produzidos um cenário aprovado e uma falha controlada, além de uma aplicação web conteinerizada e testes automatizados.

Objetivo geral

Compreender e simular o funcionamento essencial de um pipeline de entrega, observando a relação entre versionamento, jobs, imagens Docker, testes e critérios de aprovação.

Objetivos específicos

Criar um repositório Git; estruturar o arquivo .gitlab-ci.yml; definir jobs de validação, teste, construção e entrega; executar o fluxo completo; simular uma falha; e analisar o comportamento do pipeline.

2. MÉTODOS E AMBIENTE

A implementação foi organizada como um projeto denominado **UNOPAR_CI_CD**. Como o Git, o Python e o Docker já estavam instalados e funcionais, não foi necessária a instalação adicional de ferramentas. A execução local reproduziu a ordem e as condições descritas no arquivo do GitLab.

Componente	Configuração utilizada
Sistema operacional	macOS em arquitetura Apple Silicon
Git	2.54.0 (Apple Git-157)
Python	3.9.6 para testes e validação local
Docker	29.1.3 para construção da imagem
Aplicação	Loja Virtual estática responsiva denominada AgileStore
Pipeline	Validate, Test, Build e Deploy



Figura 1 - Fluxo de Integração Contínua implementado na prática.

A adaptação local evita dependência de rede durante a execução. O arquivo .gitlab-ci.yml continua preparado para runners do GitLab, enquanto o script pipeline-local.sh permite reproduzir os mesmos controles no computador.

3. REPOSITÓRIO GIT

O repositório foi iniciado na branch **main**. Os arquivos da aplicação, do pipeline, dos testes e das notificações foram adicionados ao histórico em commits separados, permitindo rastrear a evolução da solução.

```
git init -b main
git config user.name "Guilherme Dorigon Bovo"
git add .
git commit -m "ci: adiciona pipeline da loja virtual"
```



Figura 2 - Histórico de commits e estrutura do repositório UNOPAR_CI_CD.

A utilização de commits pequenos e descritivos facilita auditoria, reversão e investigação de falhas. O estado final da branch permaneceu limpo após a execução.

4. DEFINIÇÃO DO PIPELINE

O arquivo .gitlab-ci.yml define a sequência de estágios e os jobs associados. Cada job contém uma cláusula **script** e utiliza uma imagem compatível com a tarefa.

```
stages:
  - validate
  - test
  - build
  - deploy

variables:
  IMAGE_NAME: "loja-virtual-ci:${CI_COMMIT_SHORT_SHA}"

validar_yaml_e_codigo:
  stage: validate
  image: python:3.12-alpine
  script:
    - python -m py_compile scripts/validate_app.py
    - python scripts/validate_app.py

testar_aplicacao:
  stage: test
  image: python:3.12-alpine
  script:
    - python -m unittest discover -s tests -v
```

Os estágios impõem ordem ao fluxo. Por padrão, todos os jobs de um estágio devem ser aprovados antes que o estágio seguinte seja iniciado. Essa dependência impede que um artefato com erro seja construído ou entregue.

4.1 Jobs e estágios

Estágio	Job	Responsabilidade	Critério de aprovação
Validate	validar_yaml_e_codigo	Confere sintaxe, arquivos e marcadores obrigatórios.	Validador termina com código 0.
Test	testar_aplicacao	Executa três testes automatizados sobre HTML e CSS.	Todos os testes retornam OK.
Build	construir_imagem	Empacota a Loja Virtual em uma imagem Docker.	Docker build gera identificador SHA.
Deploy	entrega_simulada	Publica o artefato e verifica disponibilidade.	Servidor responde HTTP 200.

Construção e entrega

```
construir_imagem:
  stage: build
  image: docker:27-cli
  services:
    - docker:27-dind
  script:
    - docker build -t "$IMAGE_NAME" .

entrega_simulada:
  stage: deploy
  script:
    - echo "Imagem validada e pronta para entrega controlada."
    - sh scripts/notificacaoSucesso.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

A regra de entrega restringe o job à branch main. Em um ambiente real, outras proteções poderiam exigir revisão, aprovação manual e credenciais protegidas.

5. EXECUÇÃO COM SUCESSO

A primeira execução percorreu todos os estágios. O validador confirmou os arquivos, os três testes retornaram OK, o Docker gerou a imagem e a aplicação respondeu com código HTTP 200.

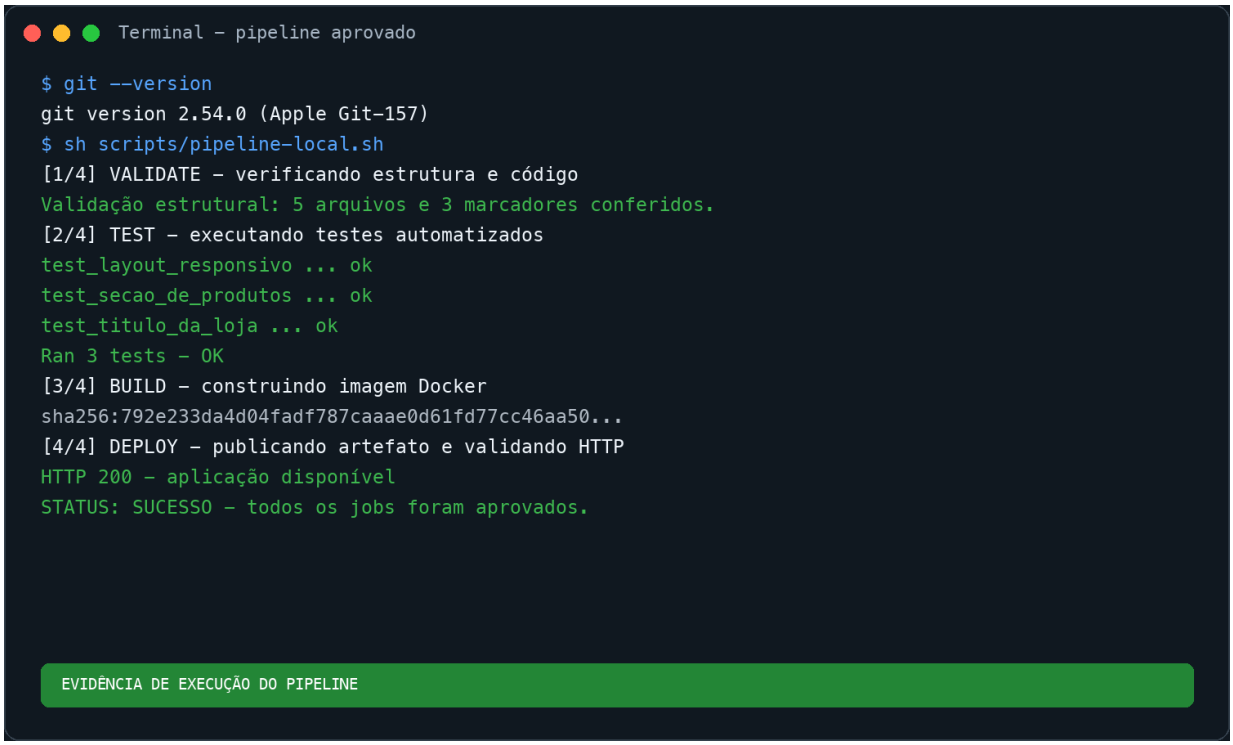


Figura 3 - Pipeline aprovado após validação, testes, build e verificação HTTP.

Verificação	Resultado
Estrutura	5 arquivos e 3 marcadores conferidos
Testes	3 executados, 0 falhas
Imagem Docker	Construída com identificador SHA-256
Disponibilidade	HTTP 200
Status final	SUCESSO

5.1 Build e entrega

O Dockerfile gera uma imagem de artefato estático. O uso de uma base **scratch** na simulação local evita downloads durante a prática e mantém apenas os arquivos necessários da aplicação. No GitLab CI, o job pode ser ajustado para publicar a imagem em um registry e executar um contêiner com servidor web.

```
FROM scratch

LABEL org.opencontainers.image.title="AgileStore"
LABEL org.opencontainers.image.description="Artefato estático da Loja Virtual"

COPY public/ /app/public/
```

Validação da entrega local

```
python3 -m http.server 8091 --directory public
curl -fsS http://127.0.0.1:8091/health.html
curl -fsS -o /dev/null -w "HTTP %{http_code}" http://127.0.0.1:8091/

# Resultado observado: HTTP 200
```

A verificação HTTP funciona como um smoke test: ela confirma que o artefato publicado pode ser acessado. Caso o servidor não responda no tempo previsto, o pipeline termina com erro e não registra a entrega como aprovada.

Rastreabilidade

O nome da imagem utiliza o identificador curto do commit. Essa prática associa cada artefato ao código que o originou e reduz ambiguidades durante auditorias, rollback e investigação de incidentes.

6. SIMULAÇÃO DE FALHA

O segundo cenário utilizou a variável **FORCE_FAIL=1** para simular uma reprovação após o job de teste. O processo retornou código de saída 1, exibiu a notificação de falha e interrompeu os estágios Build e Deploy.



Figura 4 - Falha controlada com interrupção do pipeline antes da construção e da entrega.

O comportamento observado representa a característica central da Integração Contínua: um erro em estágio anterior bloqueia as etapas dependentes. Assim, um commit reprovado não gera um artefato considerado válido e não deve alcançar o ambiente de produção.

```
FORCE_FAIL=1 sh scripts/pipeline-local.sh
# STATUS: FALHA
# exit code: 1
# BUILD e DEPLOY não executados
```

7. APLICAÇÃO RESULTANTE

A aplicação utilizada no pipeline foi uma Loja Virtual responsiva denominada AgileStore. A página apresenta produtos e uma seção que resume as quatro etapas do próprio pipeline, permitindo relacionar o artefato final ao processo que o validou.

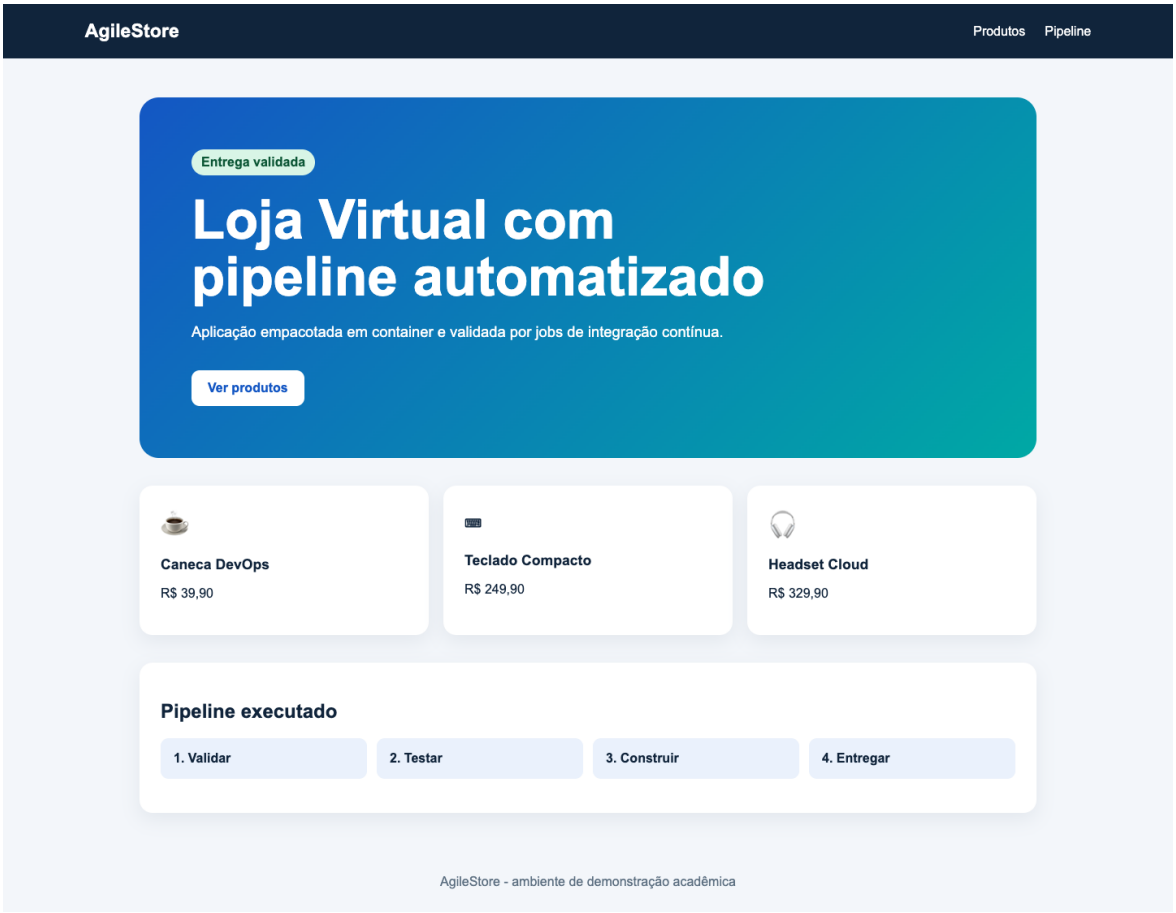


Figura 5 - Loja Virtual executada localmente após aprovação do pipeline.

A captura confirma que os arquivos empacotados foram publicados corretamente. O layout utiliza HTML semântico, CSS responsivo e uma rota de healthcheck dedicada para monitoramento automatizado.

8. ANÁLISE DOS RESULTADOS

A prática demonstrou o ciclo completo entre alteração versionada e artefato validado. O Git forneceu histórico e identificação dos commits; o arquivo YAML declarou a automação; os testes reduziram o risco de regressão; o Docker produziu um pacote imutável; e a verificação HTTP confirmou a publicação.

Aspecto	Evidência observada
Versionamento	Branch main e três commits descritivos.
Automação	Quatro estágios definidos no .gitlab-ci.yml.
Qualidade	Três testes automatizados aprovados.
Reprodutibilidade	Imagem Docker identificada por SHA-256.
Monitoramento	Smoke test HTTP 200 após publicação.
Controle de falha	Código 1 interrompeu Build e Deploy.

Segurança e confiabilidade

A configuração não contém senhas ou tokens no repositório. Em uma implantação real, credenciais do registry e do ambiente devem ser armazenadas como variáveis protegidas do GitLab. A branch de produção deve exigir revisão, e imagens devem ser analisadas quanto a vulnerabilidades.

Desempenho

Os testes atuais são rápidos por avaliarem arquivos estáticos. Em projetos maiores, cache de dependências e execução paralela podem reduzir o tempo do pipeline. Entretanto, a otimização nunca deve remover verificações essenciais.

Melhorias futuras

Publicar a imagem em um registry privado; executar scanner de vulnerabilidades; assinar artefatos; adicionar testes end-to-end; configurar ambientes de homologação; implementar aprovação manual para produção; e registrar métricas de duração e taxa de falhas.

9. CONCLUSÃO

A atividade atingiu o objetivo de simular o monitoramento de um pipeline de entrega utilizando Git. O repositório foi estruturado com os arquivos necessários, o .gitlab-ci.yml definiu a ordem dos jobs e a aplicação foi submetida a validação, testes, construção e entrega simulada.

No cenário aprovado, todos os jobs foram concluídos, uma imagem Docker foi construída e a Loja Virtual respondeu com HTTP 200. No cenário de falha, o código de saída diferente de zero interrompeu corretamente as etapas seguintes. Essa comparação tornou evidente como o pipeline protege o fluxo de entrega.

Conclui-se que a Integração Contínua aumenta a confiabilidade ao transformar verificações repetitivas em controles automáticos e rastreáveis. Git, GitLab CI/CD e Docker atuam em conjunto para relacionar código, execução, resultado e artefato.

Referências utilizadas

ANHANGUERA. *Roteiro de Aula Prática: Infraestrutura Ágil*. Material didático fornecido na disciplina.

GIT. *Documentação do sistema de controle de versões distribuído*.

GITLAB. *Documentação de CI/CD e sintaxe do arquivo .gitlab-ci.yml*.

DOCKER. *Documentação de construção e gerenciamento de imagens*.

Prática executada localmente e evidências consolidadas em 17 de setembro de 2026.